

Transfer learning, Object Localization and Object Detection

Object Detection and Basic Object Detection Algorithms

Some of the Building Blocks for Object Detection

Courtesy: Andrew NG

What is Object localization and Detection?

- Object Detection
- Object Localization

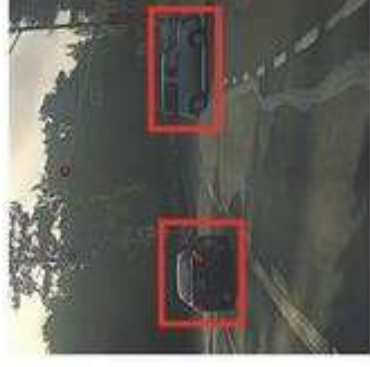
Image classification



Classification with localization



Detection



Classification with Localization

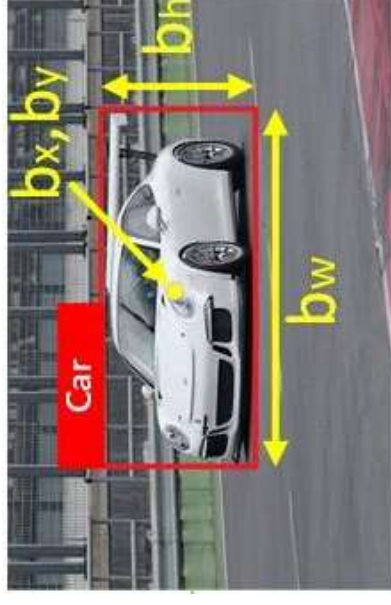


- Object categories for self-driving car

1. Car 2. Bike 3. Truck 4. Background

- Softmax with 4 possible outputs and

4 output units for bounding box (b_x, b_y, b_h, b_w)



How to define the target label Y

- 4 Object categories: 1. Car 2. Bike, 3. Truck, 4. Background
- Need to output b_x , b_y , b_h , and class labels (1-4)



$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b^0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

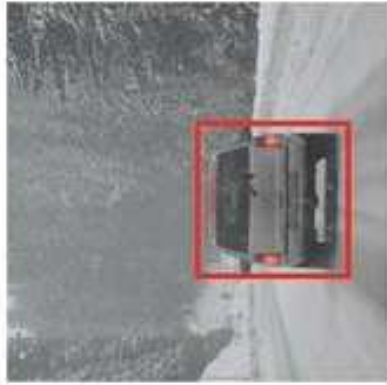


$$\begin{bmatrix} -b^x & b^y & b^h & b^0 & -0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$L(\hat{y}, y) = \begin{cases} (\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 \\ + \dots + (\hat{y}_8 - y_8)^2 & \text{if } y_1 = 1 \\ (\hat{y}_1 - y_1)^2 & \text{if } y_1 = 0 \end{cases}$$

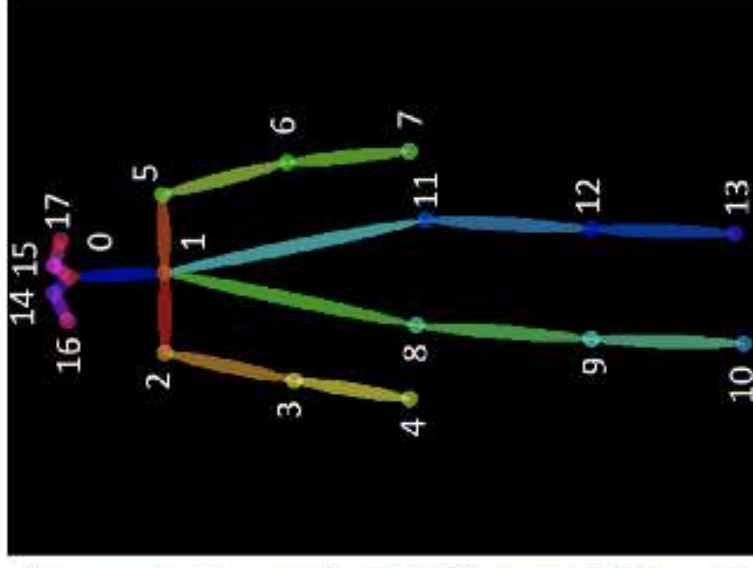
Landmark Detection



b_x, b_y, b_h, b_w

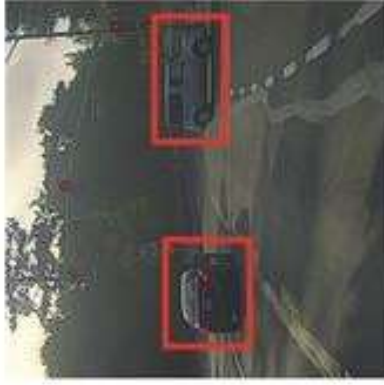


$l_{1x}, l_{1y},$
 $l_{2x}, l_{2y},$
 $l_{3x}, l_{3y},$
 $l_{4x}, l_{4y},$
 $\vdots$
 l_{64x}, l_{64y}

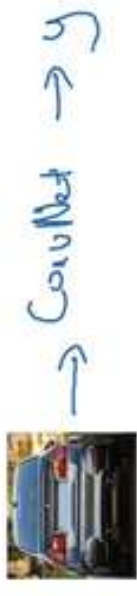


Object Detection

- Can we use Convolution Neural Net for Object Detection?
- Object Detection algorithm: Conv. Neural Net with sliding windows
- Car detection example



Training set:



Sliding windows Detection (for Object detection)

- The trained ConvNet can be used in sliding windows detection. (Slide window across the entire image)



→ ConvNet → 0

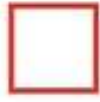


→ ConvNet → 0



...

→ ConvNet → 0



Sliding windows Detection (for Object detection)

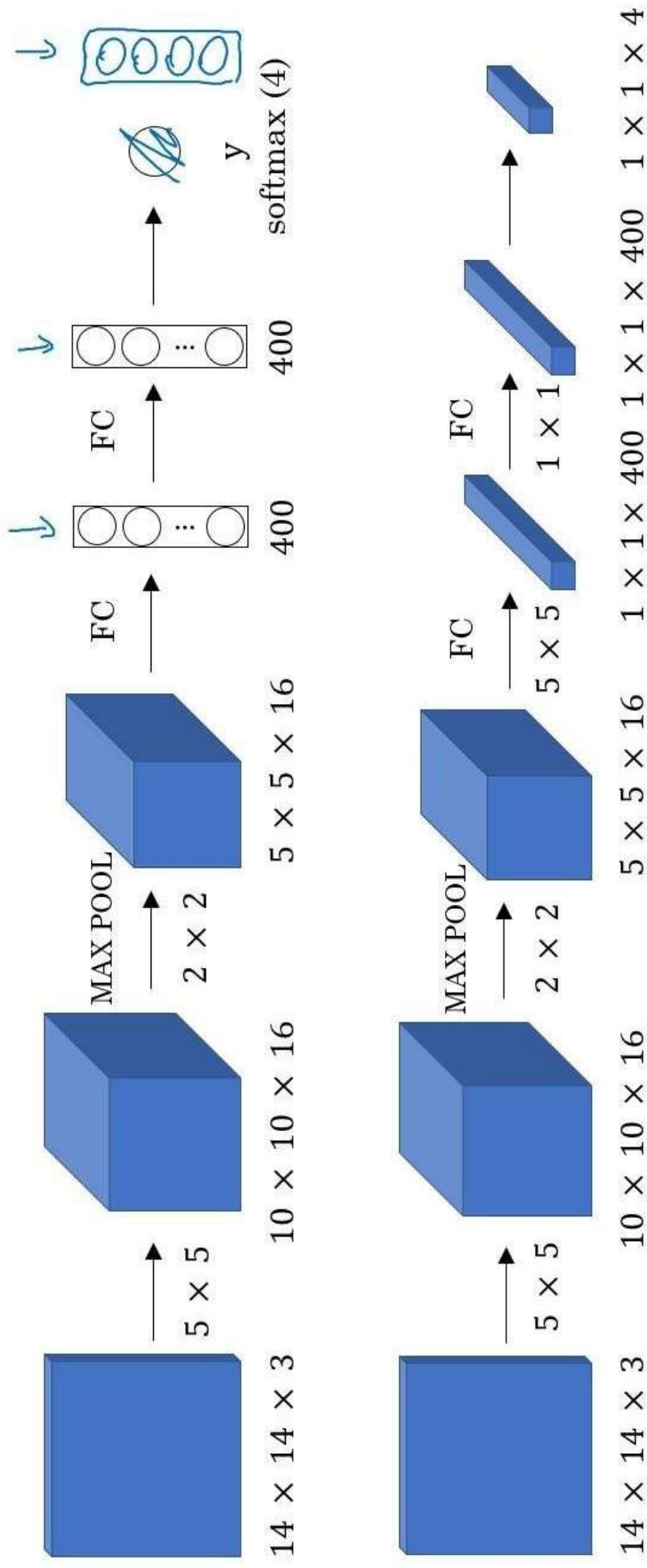
- Sliding window with Larger size



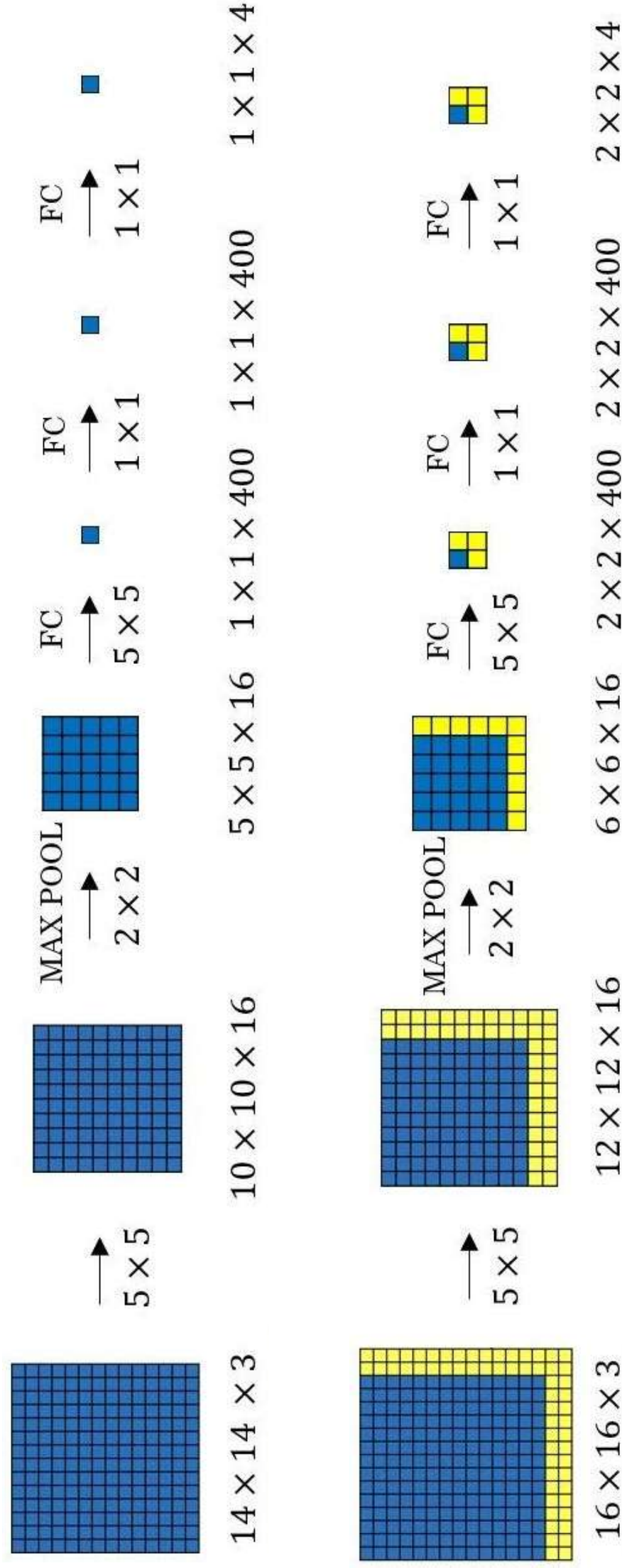
Convolution Implementation of Sliding Windows

Convolution Implementation of Sliding Windows

- Fully connected layers are concerted in convolution layers

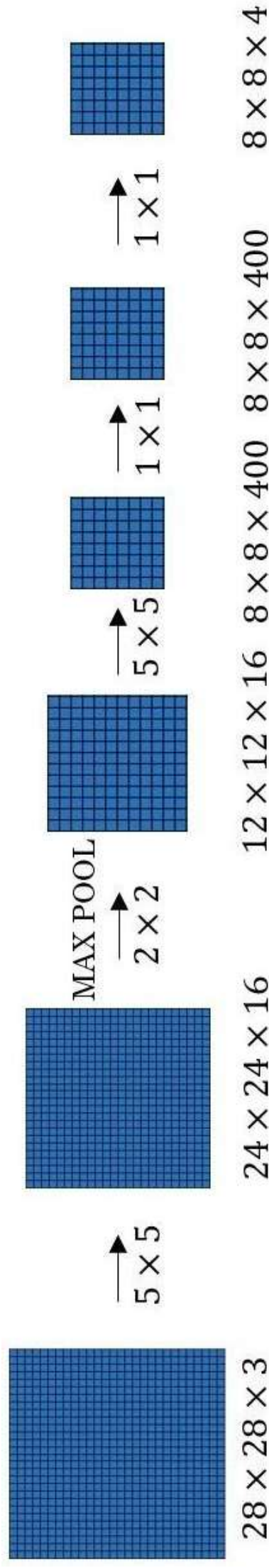


Convolution Implementation of Sliding Windows



[Sermanet et al., 2014, OverFeat: Integrated recognition, localization and detection using convolutional networks]

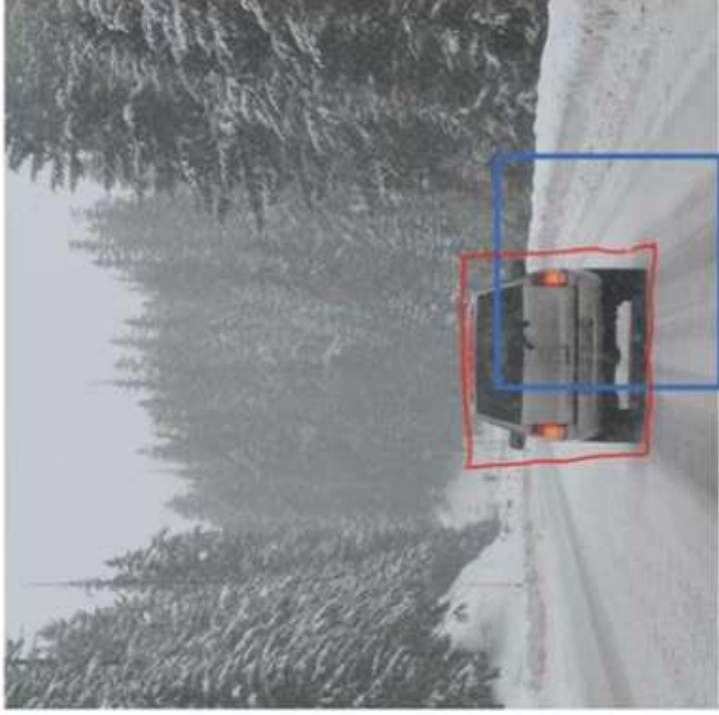
Convolution Implementation of Sliding Windows



[Sermanet et al., 2014, OverFeat: Integrated recognition, localization and detection using convolutional networks]

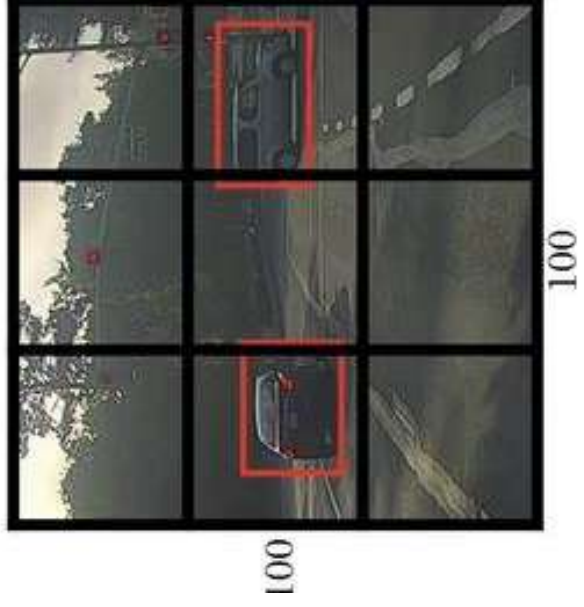
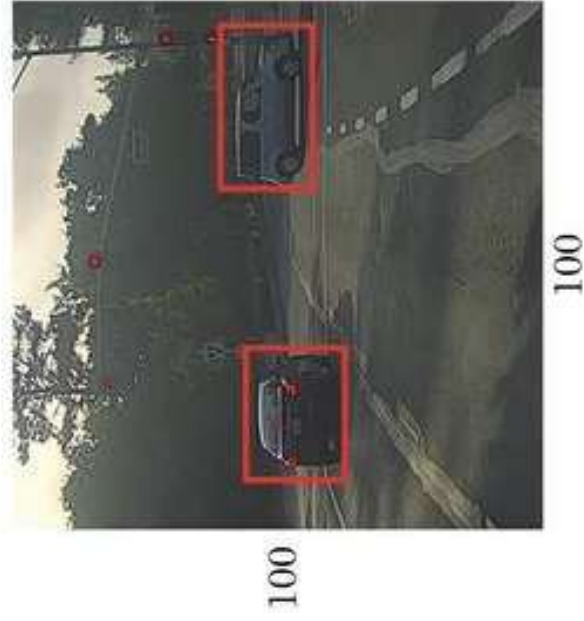
Convolution Implementation of Sliding Windows

- Weakness:
 - Bounding boxes may not be really accurate



Accurate Object Detection (Bounding box prediction) using YOLO Algorithm

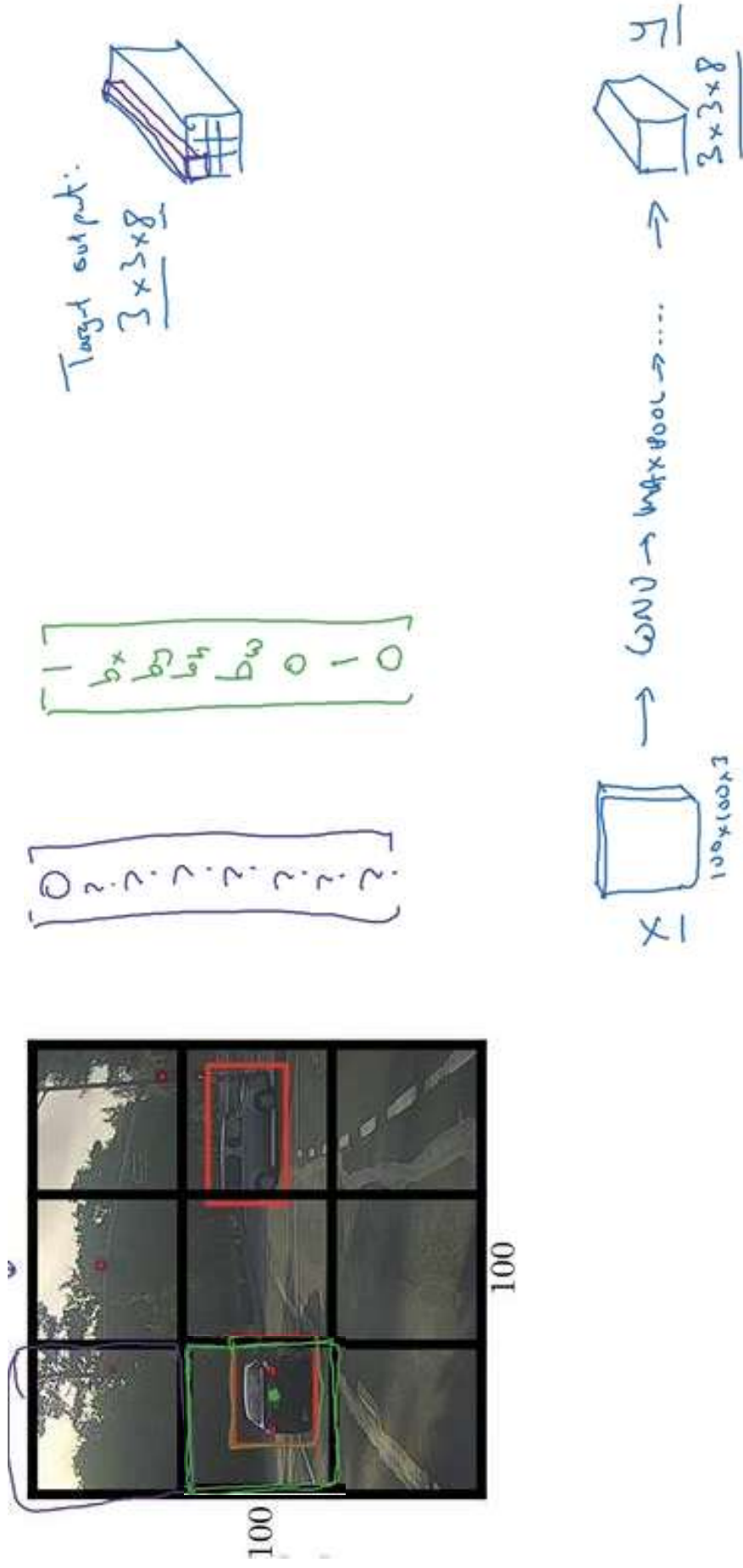
YOLO Algorithm



Labels for training
For each grid cell:

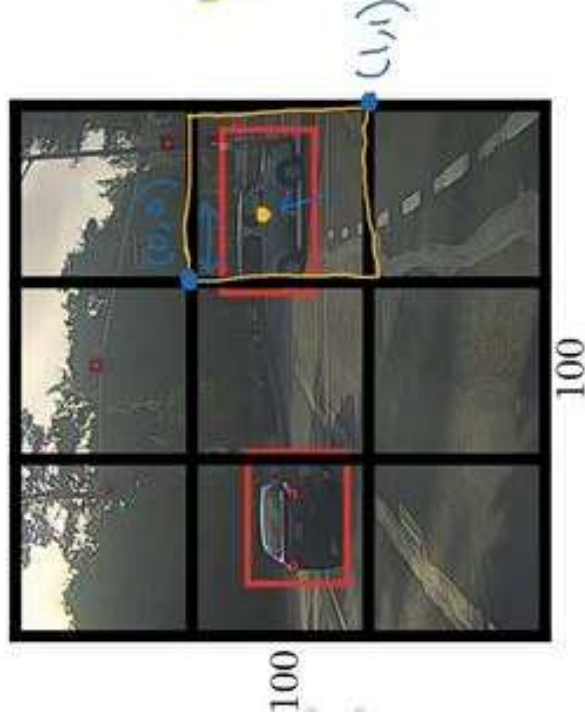
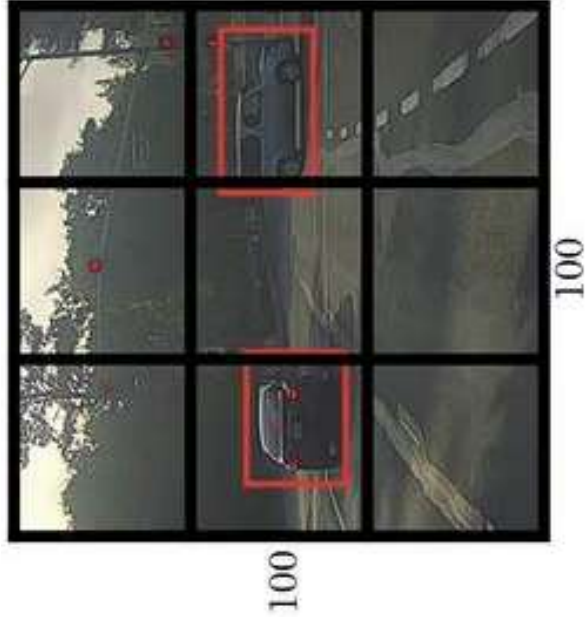
$$y = \begin{matrix} p_c & b_x & b_y & b_h & b_w & c_1 & c_2 & c_3 \end{matrix}$$

YOLO Algorithm



[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

How to specify the Bounding box?

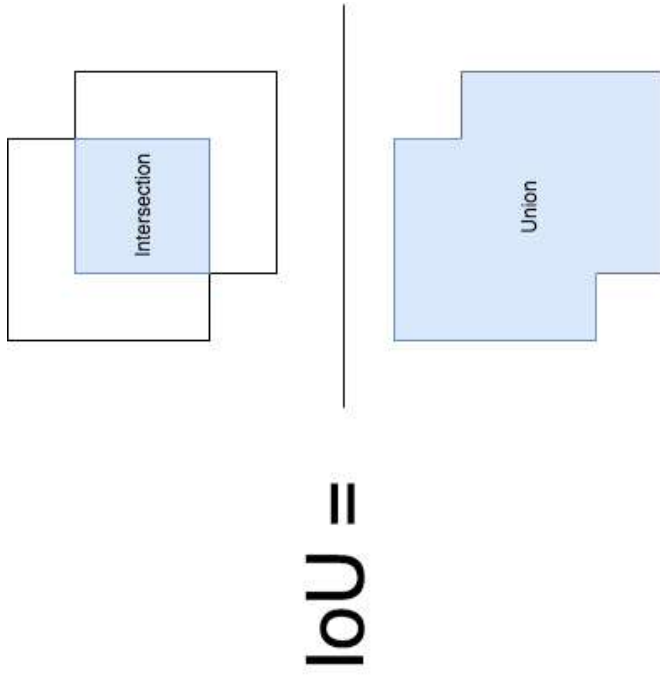
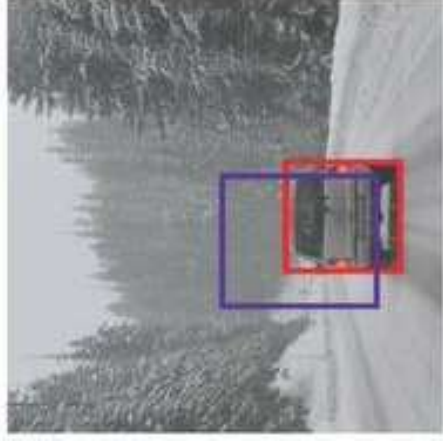


$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{array}{l} 0.4 \\ 0.3 \end{array} \left. \begin{array}{l} \text{between} \\ \text{0 and 1} \end{array} \right\} \begin{array}{l} \text{could be} \\ > 1 \end{array}$$

Evaluating Object Localization

Intersection Over Union (IoU): It is a ratio between the **intersection** and the **union** of the **predicted** boxes and the **ground truth** boxes. This stat is also known as the Jaccard Index and was first published by Paul Jaccard in the early 1900s.



Often used: correct if $\text{IoU} \geq 0.5$

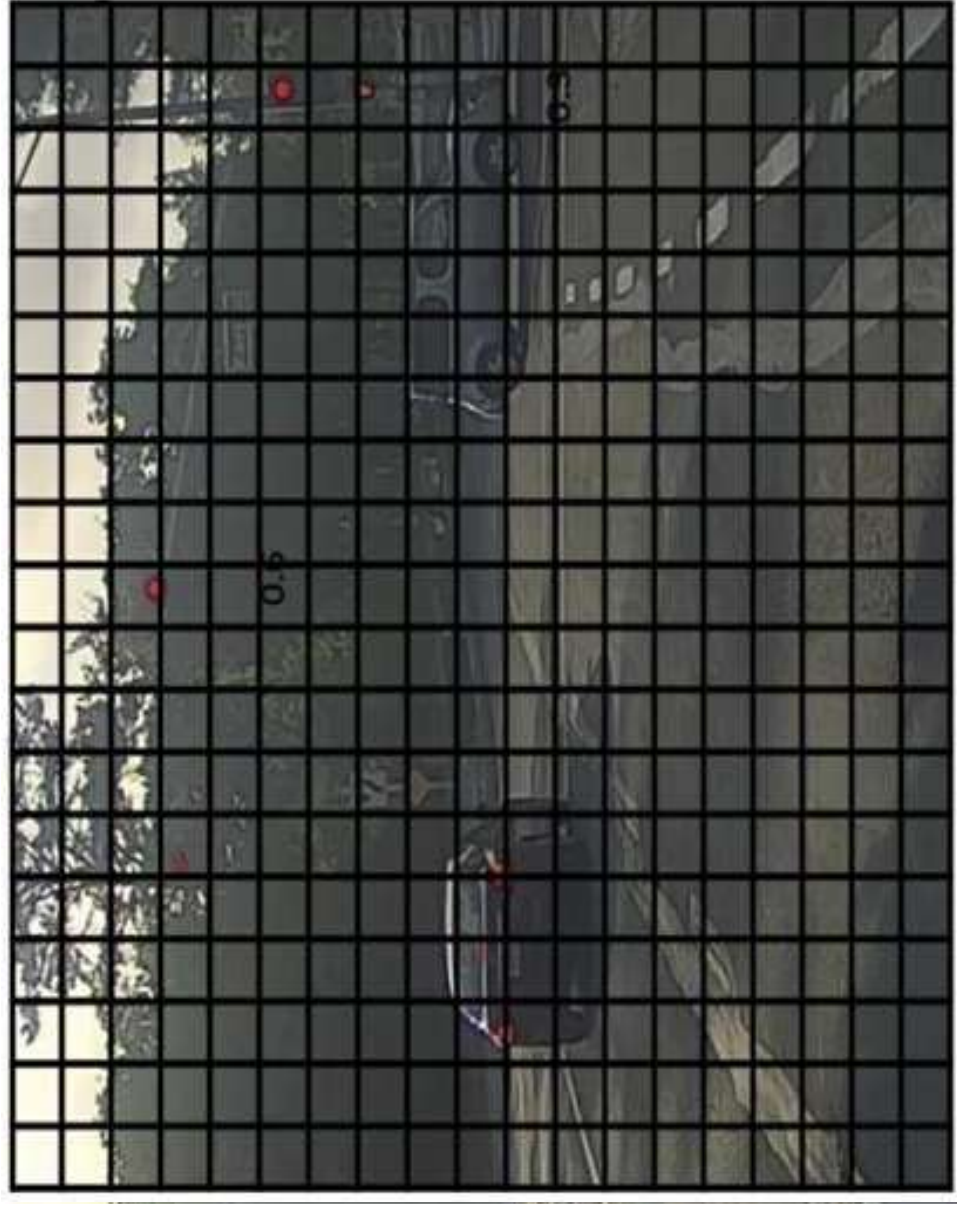
More generally, IoU is a measure of the overlap between two bounding boxes.

Object Detection:

Non-max Suppression

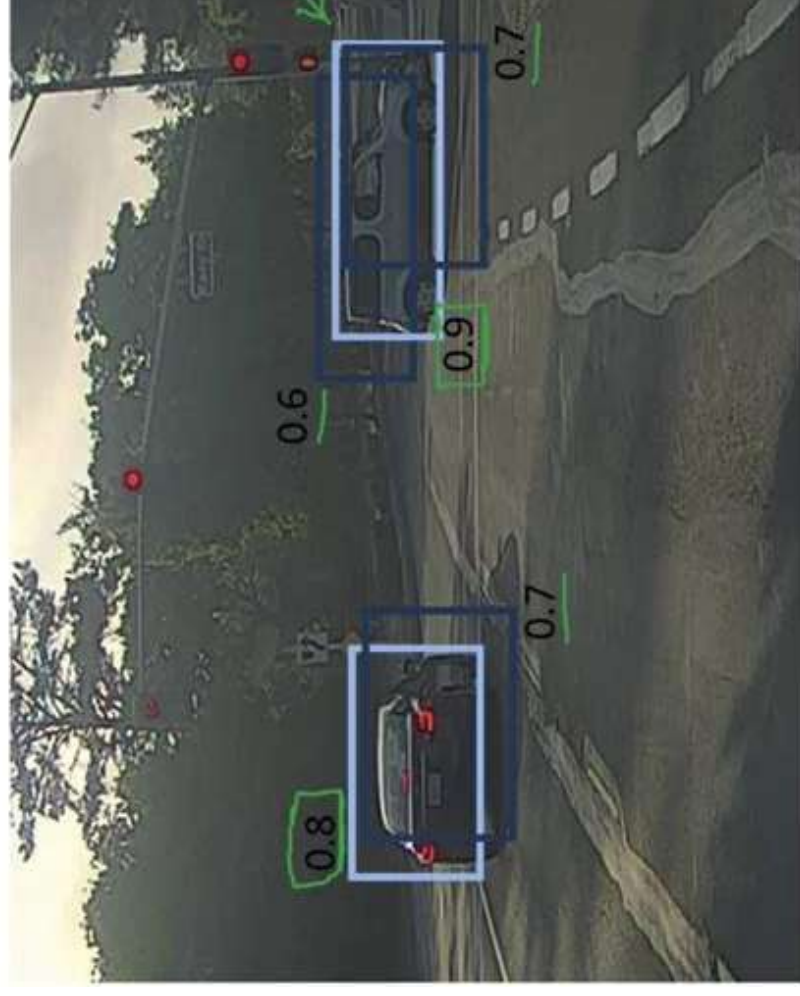
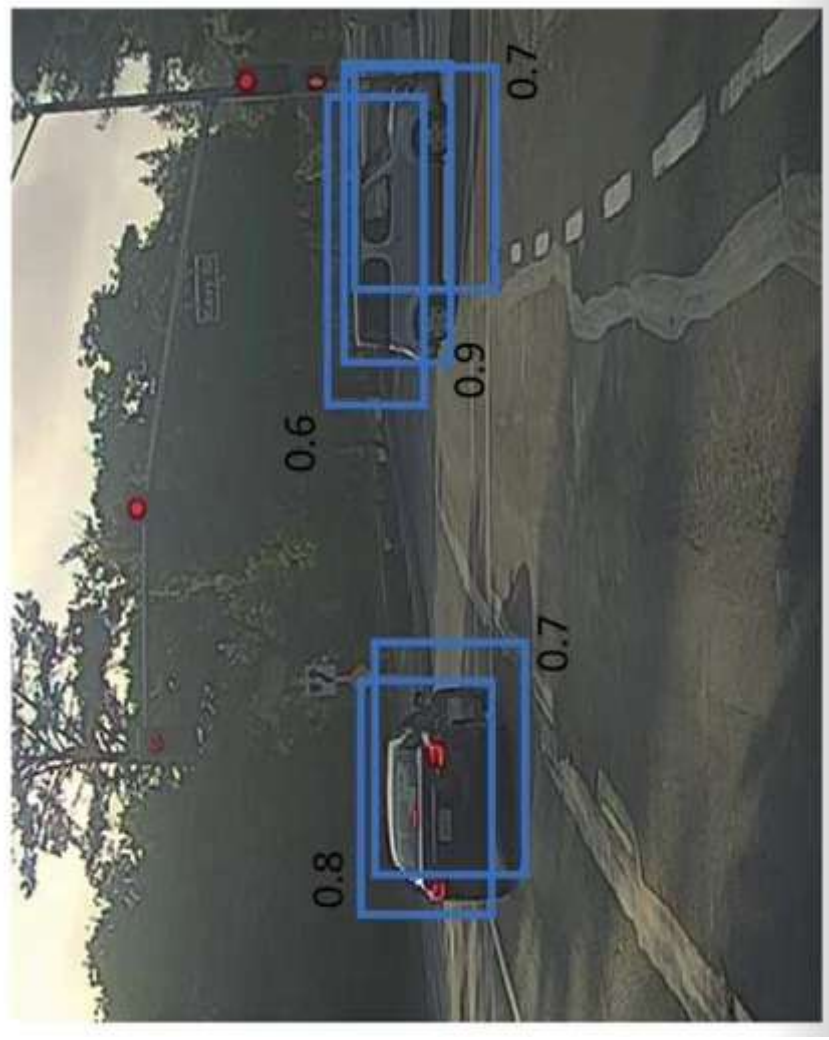
Non-max suppression

Example:



Non-max suppression

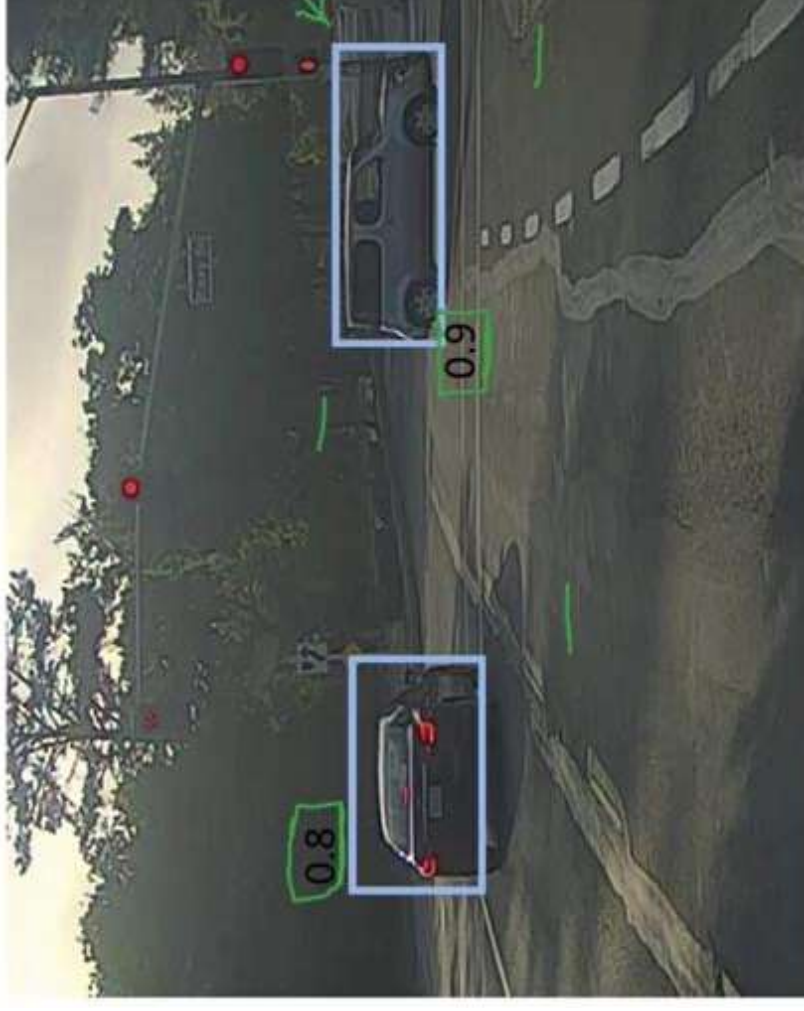
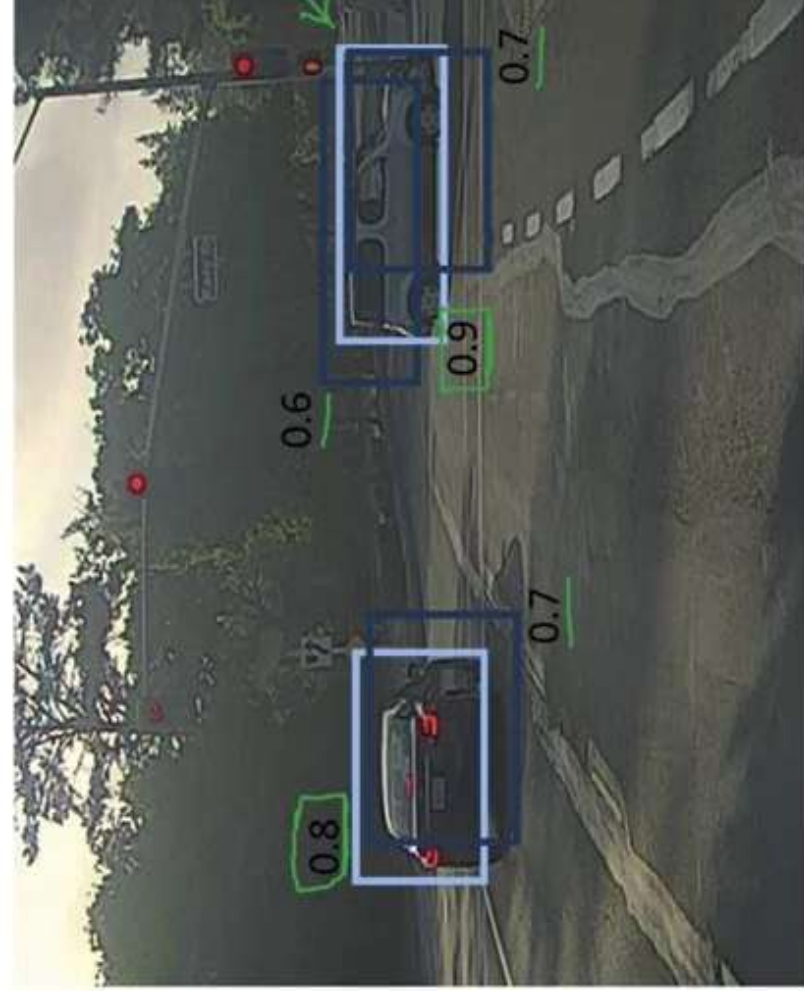
Example:



[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Non-max suppression

Example:



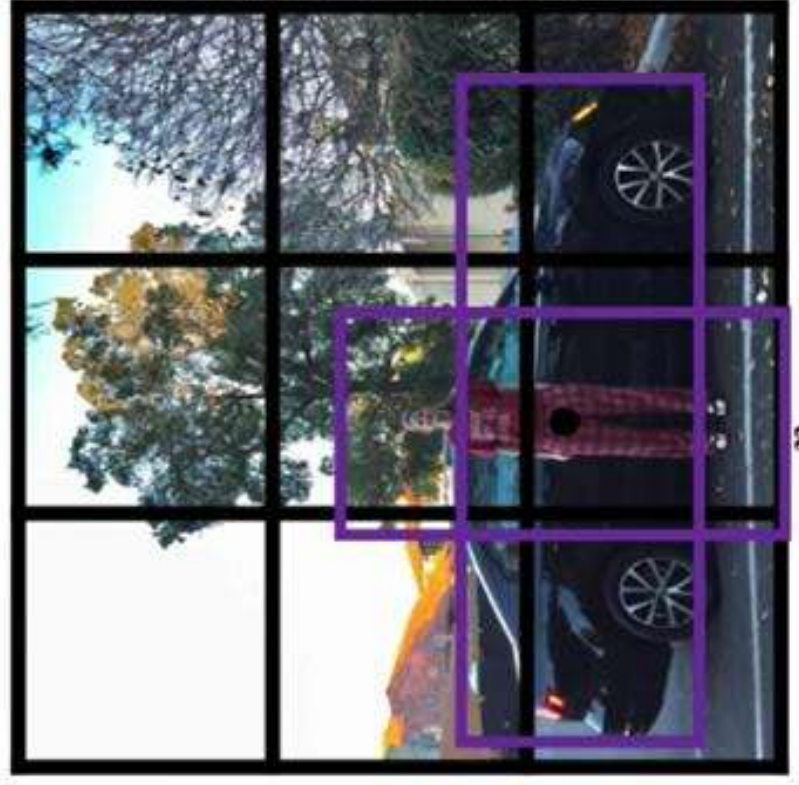
[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Object Detection:

Anchor Boxes

Anchor Boxes

Overlapping Objects



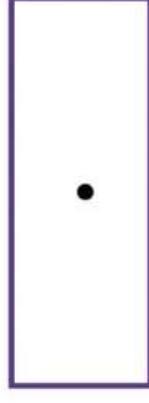
$y =$

$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Anchor box 1:



Anchor box 2:



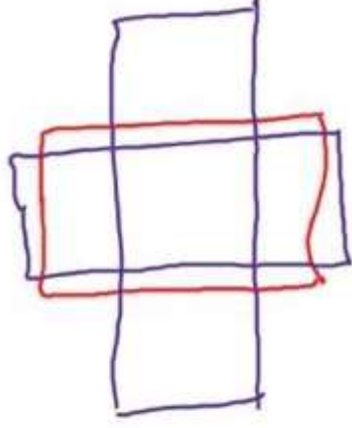
$$y = \begin{bmatrix} p_c & b_x & b_y & b_h & b_w & c_1 & c_2 & c_3 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

Anchor box 1 Anchor box 2

Anchor Box Algorithm

Previously:

Each object in training image is assigned to grid cell that contains that object's midpoint.

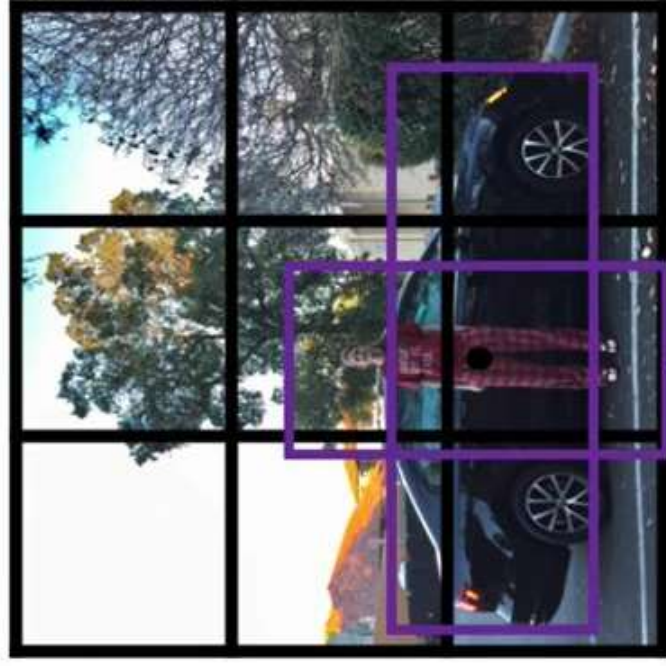


(Grid cell, anchor box)

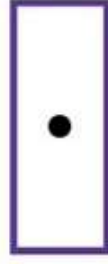
With two anchor boxes:

Each object in training image is assigned to grid cell that contains object's midpoint and anchor box for the grid cell with highest IoU.

Anchor Box Example



Anchor box 1: Anchor box 2:



$y =$

$$\begin{bmatrix} p_c & b_x & b_y & b_h & b_w & c_1 & c_2 & c_3 & p_c & b_x & b_y & b_h & b_w & c_1 & c_2 & c_3 \end{bmatrix}$$

$$\begin{bmatrix} b_x & b_y & b_h & b_w & 0 & 0 & b_x & b_y & b_h & b_w & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & b_x & b_y & b_h & b_w & 0 & 0 \end{bmatrix}$$

Anchor box 1
Anchor box 2

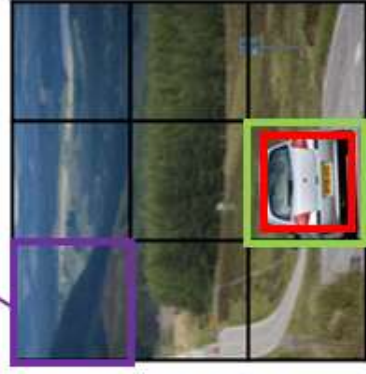
Overall: YOLO Object Detection Algorithm

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

YOLO Algorithm

Training

1. Pedestrian
2. Car
3. Motorcycle



$$y = 3 \times 3 \times 16$$

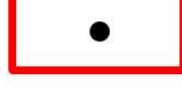
$$y = 3 \times 3 \times 2 \times 8$$

anchors # parameters

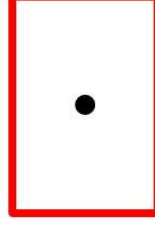
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

$$y = \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Anchor box 1:

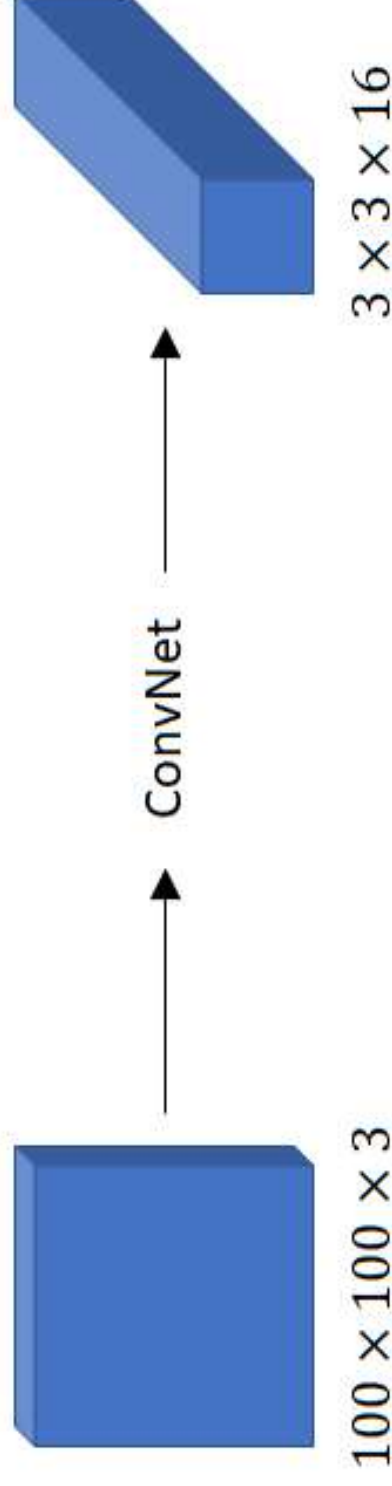


Anchor box 2:



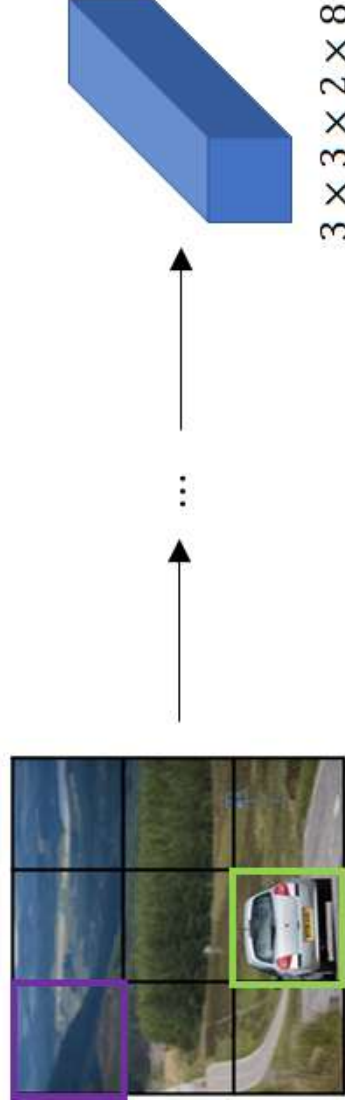
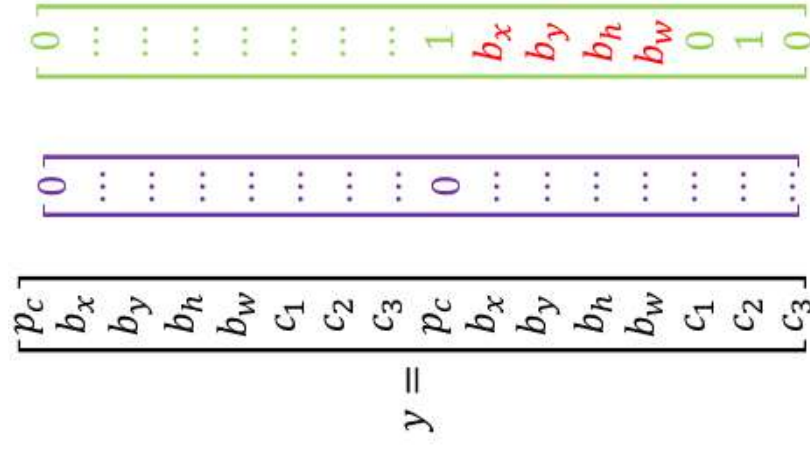
YOLO Algorithm

Training



YOLO Algorithm

Prediction



[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

YOLO Algorithm

Prediction: How do we run a non-max suppression?

- For each cell get 2 predicted bounding boxes
- Get rid of low probability predictions
- For each class (pedestrian, car, motorbike), use non-max suppression to generate final predictions.